

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Компьютерная Графика»
Тема: Формирование различных кривых с использованием
ортогонального проектирования на плоскость визуализации (экране
дисплея)

Студенты гр. 3311	_____	Землякова С. А.
	_____	Кудрин П. В.
	_____	Сапронов К. Д.
Преподаватель	_____	Колев Г.Ю.

Санкт-Петербург
2025

Цель работы

Сформировать на плоскости кривую Безье на основе задающей ломаной, определяемой 3 и большим количеством точек. Обеспечить редактирование координат точек задающей ломаной с перерисовкой сплайна Безье.

Теоретическая часть

В данной работе кривая Безье строится на основе набора контрольных точек, которые определяют ее форму. Если соединить эти точки линиями, то получится задающая ломаная. Кривая Безье на основе этой ломаной строится таким образом, что она проходит через первую и последнюю контрольные точки и следует общей форме ломаной.

Кривая Безье строится на основе набора контрольных точек $P_0, P_1, \dots, P_n$, которые образуют задающую ломаную. Особенность кривой Безье заключается в том, что она всегда проходит через первую и последнюю контрольные точки, а ее форма плавно следует направлению задающей ломаной.

Для построения кривой Безье был использован рекурсивный алгоритм де Кастельо, который обладает наглядной геометрической интерпретацией и вычислительной эффективностью.

Для заданного параметра $t \in [0,1]$ точка на кривой Безье вычисляется по рекуррентной формуле: $P_i^{(r)}(t) = (1 - t) \cdot P_i^{(r-1)}(t) + t \cdot P_{i+1}^{(r-1)}(t)$, где:

- $r = 1, 2, \dots, n$ (уровень рекурсии)
- $i = 0, 1, \dots, n - r$
- $P_i^{(0)} = P_i$ (исходные контрольные точки)

На каждом шаге алгоритма между соседними точками текущего уровня строится линейная интерполяция с параметром t . Процесс повторяется рекурсивно до тех пор, пока не останется одна точка - искомая точка на кривой Безье.

Описание интерфейса

Основные элементы управления:

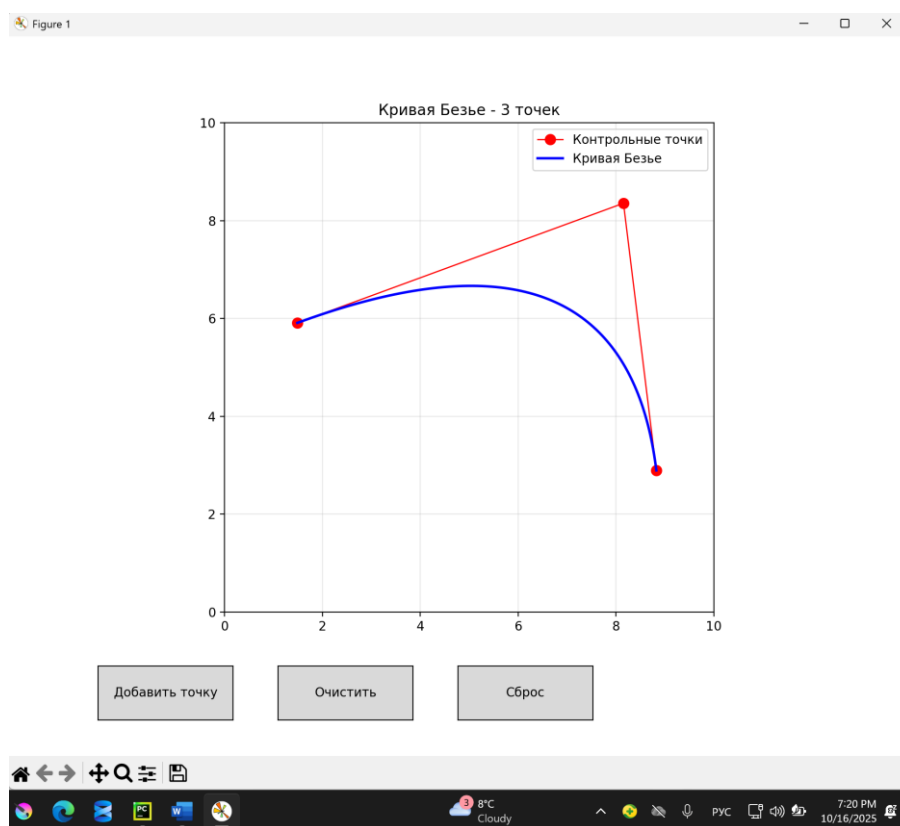
1. Область построения с декартовой системой координат
2. Интерактивное добавление контрольных точек щелчком мыши
3. Перетаскивание существующих точек для изменения формы кривой
4. Удаление точек правым щелчком мыши
5. Панель управления с кнопками для дополнительных операций

Функциональные возможности:

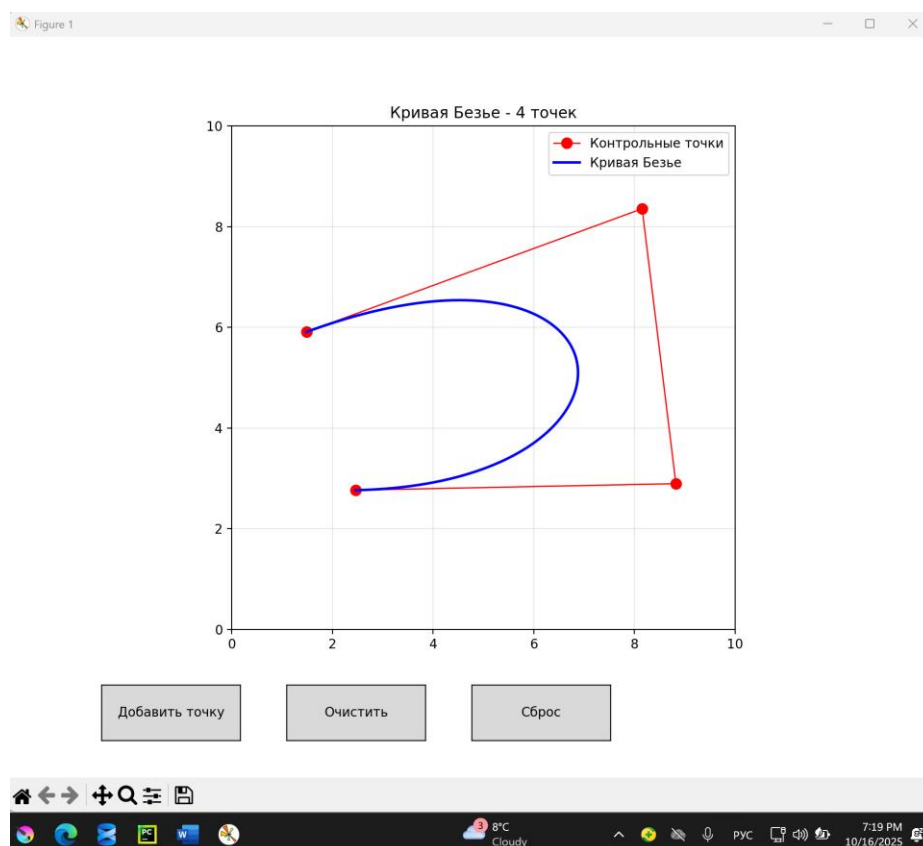
1. Автоматическое обновление кривой при изменении контрольных точек
2. Визуализация как кривой Безье, так и задающей ломаной
3. Отображение количества контрольных точек
4. Возможность сброса к начальному состоянию

Примеры работы программы.

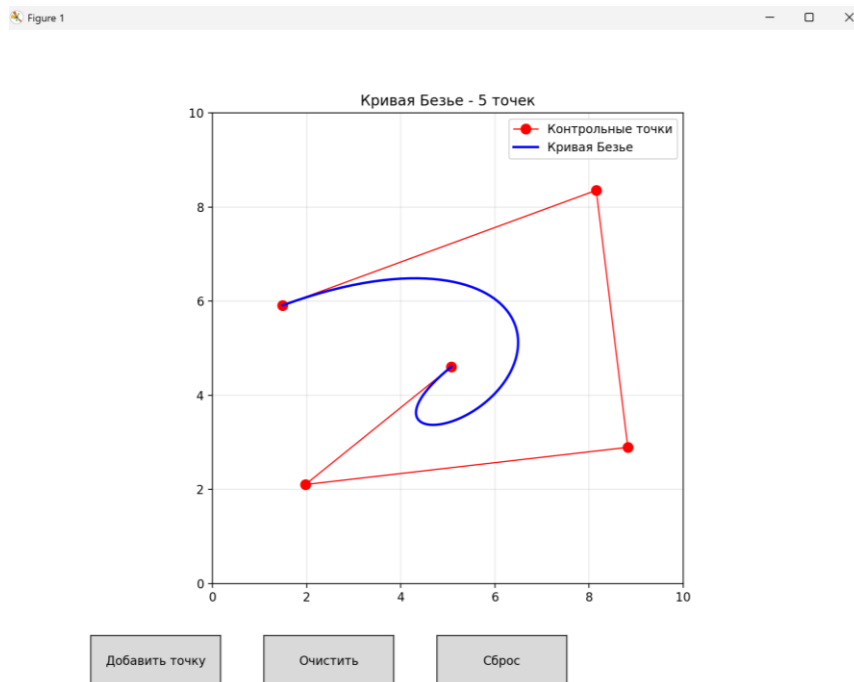
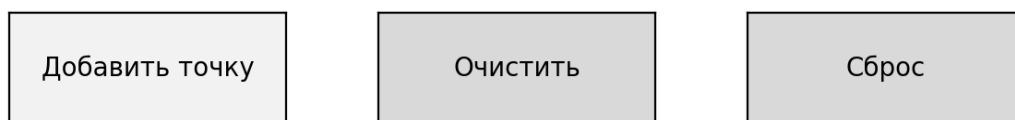
1. Ломаная на 3 точках



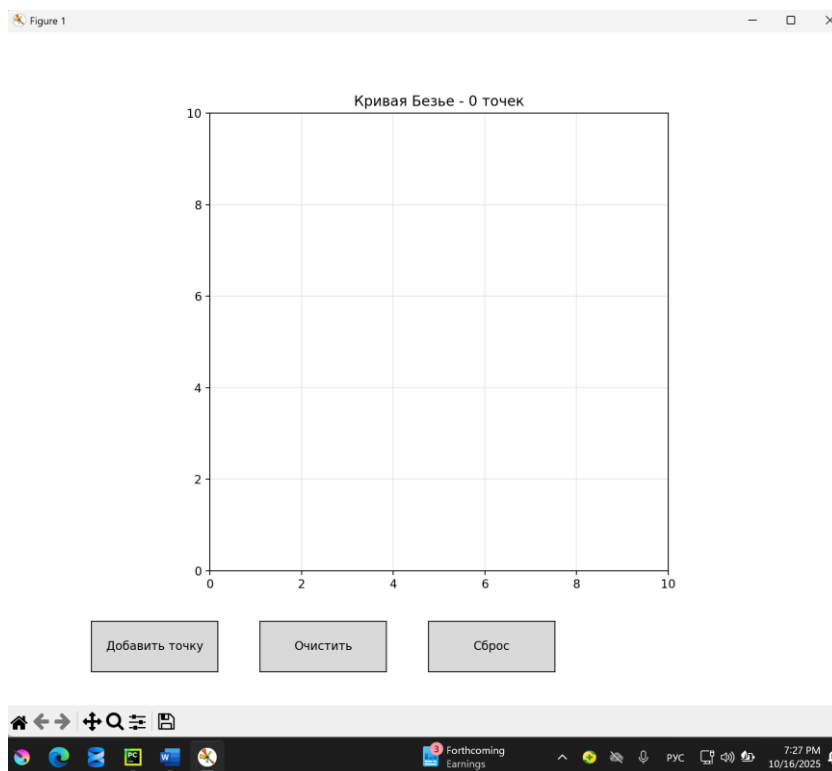
2. Ломаная на 4 точках



3. Кнопка интерфейса «Добавить точку» - добавляет случайную точку ломаной



4. Кнопка интерфейса «Очистить» - очищает экран (удаляет все существующие точки ломаной)



Вывод.

В ходе лабораторной работы была разработана программа для построения кривых Безье на плоскости. Основная цель работы - изучение алгоритма де Кастельо и его практическая реализация.

Программа позволяет интерактивно работать с кривыми: добавлять, перемещать и удалять контрольные точки, наблюдая в реальном времени изменение формы кривой. Реализованный алгоритм строит гладкие кривые, которые проходят через первую и последнюю точки и следуют направлению задающей ломаной.

Практическая работа подтвердила теоретические свойства кривых Безье: аффинную инвариантность, принадлежность «выпуклой оболочке» и гладкость.

Исходный код.

1. Функция построения кривой Безье по заданным точкам

```
def bezier_curve(self, points, num_points=100):  
    if len(points) < 2:  
        return np.array([])  
  
    n = len(points) - 1  
    t = np.linspace(0, 1, num_points)  
  
    curve = np.zeros((num_points, 2))  
    for i in range(num_points):  
        curve[i] = self.de_casteljau(points, t[i])  
  
    return curve
```

2. Алгоритм де Кастельжо

```
def de_casteljau(self, points, t):  
    if len(points) == 1:  
        return points[0]  
  
    new_points = []  
    for i in range(len(points) - 1):  
        x = (1 - t) * points[i][0] + t * points[i + 1][0]  
        y = (1 - t) * points[i][1] + t * points[i + 1][1]  
        new_points.append([x, y])  
  
    return self.de_casteljau(new_points, t)
```

3. Добавление случайной точки

```
def add_random_point(self, event):  
    """Добавляет случайную точку"""  
    x = np.random.uniform(1, 9)  
    y = np.random.uniform(1, 9)  
    self.control_points.append([x, y])  
    self.redraw()
```